Appunti del corso di
Fondamenti dell'informatica

UniVR - Dipartimento di Informatica

Primo semestre - A.A. 2025/26

Mattia Matteo Drago

Indice

1 Concetti di base **1**

1.1 Nozioni di logica 1

1.2 Nozioni di insiemistica 2

1.3 Nozioni di induzione matematica 2

2 Linguaggi formali **3**

2.1 Linguaggi regolari 3

2.2 Automa a stati finiti 4

2.2.1 Automa a stati finiti deterministico 5

2.2.2 Automa stati finiti non deterministico 5

2.2.3 Automa a stati finiti non deterministico con ϵ-transizioni 6

2.3 Espressioni regolari 7

2.3.1 Proprietà delle espressioni regolari 7

2.3.2 Proprietà di chiusura dei linguaggi 8

2.3.3 Corrispondenza tra espressioni regolari e ASFD 9

2.4 Automa minimo 9

2.4.1 Relazioni tra linguaggi 10

2.4.2 Teorema di Myhill-Nerode 11

2.5 Proprietà dei linguaggi regolari 11

3 Automi a pila **15**

1

Concetti di base

1.1 Nozioni di logica

La logica permette di avere un linguaggio (anche formale) utilizzato per descrivere i vari elementi.

Il **linguaggio del primo ordine** è un linguaggio che è formato da un certo insieme di simboli:

- *relazionali*: $p, q, r, \dots$
- *funzionali*: $f, g, h, \dots$
- *costanti*: $c, d, \dots$
- *logici*: insieme di simboli
  - parentesi tonde ( ) e virgola ,
  - insieme numerabile ($\mathbb{N}$) di variabili ($v, x, y, \dots$)
  - insieme di connettivi logici ($\wedge, \vee, \Rightarrow, \Leftrightarrow$)
  - quantificatori ($\forall, \exists$)

Mediante questi elementi, l'obiettivo è quello di definire dei termini.

Definizione: termine (induttivo)

Definiamo il concetto di **termine** in maniera induttiva:

- **caso base**: variabili, costanti
- **passo induttivo**: se f è un simbolo di funzione, $t_1, \dots, t_n$ sono termini, allora $f(t_1, \dots, f_n)$ è un termine

I termini descrivo elementi e sono alla base della costruione di **formule** che desrivono proprietà. Per descrivere il concetto di formula, bisogna introdurre il concetto di *formula atomica*.

Definizione: formula atomica

Se p è un simbolo di relazione ennario (applicabile a n termini) e $t_1, \dots, t_n$ sono termini, allora $p(t_1, \dots, t_n)$ è una formula atomica.

Definizione: formula (induttiva)

Definiamo il concetto di **formula** mediante induzione:

- **caso base**: formula induttiva
- **passo induttivo**:
  - se φ è una formula $\Rightarrow \neg\varphi$ è una formula
  - se φ e ψ sono formule $\Rightarrow \varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \Rightarrow \psi$ sono formule
  - se φ è una formula e x un variabile $\Rightarrow \forall x(\varphi)$, $\exists x(\varphi)$ sono formule

Le *equivalenze* permettono di manipolare gli oggetti che vengono utilizzati per descrivere formule sintatticamente diverse, ma lo stesso significato semantico (stesso valore di verità):

- $\neg(\neg\varphi) \equiv \varphi$
- $$\left.\begin{aligned}\neg(\varphi \wedge \psi) &\equiv (\neg\varphi) \vee (\neg\psi)\\ \neg(\varphi \vee \psi) &\equiv (\neg\varphi) \wedge (\neg\psi)\end{aligned}\right\}\text{legge di De Morgan}$$
- $(\varphi \Rightarrow \psi) \equiv \neg\varphi \vee \psi \equiv ((\neg\psi) \Rightarrow (\varphi))$ (controvariante della logica)
- $$\left.\begin{aligned}\neg(\exists x(\varphi)) &\equiv \forall x(\neg\varphi)\\ \neg(\forall x(\varphi)) &\equiv \exists x(\neg\varphi)\end{aligned}\right\}\text{legge di De Morgan}$$

Notazione

$\exists x \in S.\varphi \equiv \exists x.(x \in S \wedge \varphi)$

$(\forall x \in S.\varphi) \equiv \forall x(x \in S \Rightarrow \varphi)$

1.2 Nozioni di insiemistica

1.3 Nozioni di induzione matematica

2

Linguaggi formali

2.1 Linguaggi regolari

I **linguaggi regolari** rappresentano la famiglia dei modelli di calcolo più semplice: corrispondono al processore di un calcolatore.

I problemi relativi a questi linguaggi sono *decisionali*: una sequenza di simboli (stringa) appartiene a un linguaggio regolare, allora è risolvibile da un algoritmo e implementabile mediante una macchina stati finiti.

Definizione: stringhe

Un **simbolo** è un'entità primitiva astratta non definita formalmente (lettere e caratteri numerici).

Definizione: alfabeto

Un **alfabeto** è un'insieme di simboli denotato con Σ e avente le seguenti caratteristiche:

- **cardinalità di Σ minore di ω**:
$$|\Sigma| < \omega$$
- **insieme** $\Sigma \neq 0 \Rightarrow$ è sempre presente almeno un simbolo

L'**insieme delle stringhe** sull'alfabeto Σ è rappresentato con Σ*: insieme di tutte le sequenze di simboli che appartegono all'alfabeto Σ.

Esempio: insieme delle stringhe Σ*

Se Σ è rappresentato dall'insieme binario $\{0, 1\}$, allora la stringa 101011 appartiene a $\{0, 1\}$*:
$$\Sigma = \{0, 1\}, \quad 101011 \in \{0, 1\}^*$$

La *cardinalità* dell'insieme delle stringhe Σ*, su qualunque alfabeto, è uguale alla cardinalità dei numeri naturali:
$$|\Sigma^*| = |\mathbb{N}| = \omega$$

A partire dagli insiemi di sequenze di simboli (stringhe) si formano i **linguaggi**.

Definizione: linguaggio

Un linguaggio $\mathcal{L}$ è un sottoinisieme dell'insieme delle stringhe:

$$\mathcal{L} \subseteq \Sigma^*$$

Quanti sono i linguaggi?
Se la cardinalità di Σ* è pari alla cardinalità dei numeri naturali, allora la cardinalità dei sottoinisiemi dei numer naturali è ugale alla Cardinalità dei sottoinisiemi degli insiemi delle stringhe:

$$|\Sigma^*| = |\mathbb{N}| = \omega$$

$$\Downarrow$$

$$|2^N| = |2^{\Sigma^*}| = |\mathbb{N} \to |\mathbb{N}| = |\mathbb{R}|$$

Di conseguenza ci sono tanti linguaggi quanti i numeri reali su un alfabeto finito:

$$|\{\mathcal{L}|\mathcal{L} \subseteq \Sigma^*\}| \subseteq |\mathbb{R}|$$

2.2 Automa a stati finiti

Un **automa a stati finiti** è un dispositivo avente un nastro in *input*, suddiviso a blocchi, su cui la macchina non può scrivere, bensì solo leggere.

La stringa σ è formata dai simboli $x_0, x_1, x_2, \ldots$, mentre il **controllore** (dipositivo di lettura) parte da un certo stato q iniziale e può muoversi soltanto da sinistra verso destra.
Quindi si può leggere un simbolo x_i, consumare questo simbolo, effetture una transizione di stato passando da q a q^t e, infine, passare al simbolo sucessivo x_{i+1}.
Man mano che la testina procede, essa consuma ciò che è presente sul nastro in *input* fintatochè la stringa σ non si esaurisce.

Poichè le stringhe che si possono mettere sul nastro sono finite, allora le operazioni indicate possono continuare fino a un certo numero di passi, dopodichè la macchina termina.

Una descrizione matematicaa di questa M consiste nella quintupla:

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle$$

Q : insieme di *stati*
Σ : insieme di *simboli*
δ : funzione di *transizione*
q_0 : stato inizale da cui la macchina parte
F : insieme di stati finali

Esempio: automa a stato finito

- singola freccia entrante $\to$ *stato iniziale*
- doppio cercio $\to$ *stato finale*

- frecce $\rightarrow$ *archi* che a seconda del valore letto effettuano una transizione dal nodo cambiando lo stato

Il linguaggio accettato dall'automa è l'insieme di tutte òe stringhe sull'alfabeto tale che $\hat{\delta}(q_0, \sigma) \in F$:

$$\mathcal{L}(M) = \{\sigma \in \Sigma^* | \hat{\delta}(q_0, \sigma) \in F\}$$

Definizione: linguaggio regolare

Se M è un'automa a stati finiti deterministico, allora $\mathcal{L}$ è detto **regolare**.

2.2.1 Automa a stati finiti deterministico

2.2.2 Automa stati finiti non deterministico

Un automa a stati finiti deterministico è definito come:

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle$$

L'unico campo a cambiare è la funzione di stato sucessore δ, che negli ASFND è una **relazione**.
Quindi si ha:

$$\delta \subseteq (Q \times \Sigma) \times Q$$

Data una coppia $\Rightarrow$ la macchina è in uno stato $q \Rightarrow$ legge un simbolo $x \in \Sigma \Rightarrow$ lo stato q non cambia in un altro stato p (andando a leggere il simbolo sucessore), bensì può cambiare in **più stati sucessori** $p_1, p_2, \ldots, p_n$.
In questo modo, la macchina può scegliere in modo non deterministico quale stato sucessore prendere.

Se l'espressione $\delta \subseteq (Q \times \Sigma) \times Q$, allora si puà rappresentare δ come:

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

Ciò sifgnifica che δ è una **funzione** che va dagli stati Q per i simboli Σ (legge un simbolo dal nastro) e genera un insieme di stati 2^Q.
Se si va nelle parti, cioè in 2^Q, allora $\emptyset \subseteq Q$ e $\delta(q, a) = 0$.
Ciò significa che nell'automa non deterministico si può non fare passi e rimanere nello steso stato.
Inoltre, per ogni stato in Q e ogni simbolo letto in Σ, un'altra proprietà è quella dove la cardinalità degli stati che si può raggiungere è sempre finita:

$$\forall q \in Q, \forall a \in \Sigma : |\delta(q, a)| < \omega$$

Esempio: automa a stati finiti non deterministico

$\Rightarrow$ *Perchè non è deterministico?*
In corrispondenza del simbolo 'a' si possono abere più transizioni diverse per lo stesso stato.
Perciò questo automa dopo aver letto il simbolo 'a' continua in due strade diverse, mentre se si è nello stato q_2 e si legge 'a', allora la macchina rimane bloccata in q_2 abortendo l'esecouzione.

Se esistono più tranzioni possibili per un simbolo in lettura, allora l'automa non è deterministico.

La relazione di transizione applicata a più passi consegue nella seguente definizione di δ:

$$\begin{cases} \hat{\delta}(q, \epsilon) = \{q\} & \text{se } \sigma = \epsilon \\ \hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q,x)} \delta(p, a), & \text{se } \sigma = xa,\ a \in \Sigma \end{cases}$$

$\Rightarrow$ se non si legge nulla, si rimane in q generico
$\Rightarrow$ se si legge unsimbolo, si genera un insieme di stati: la macchina si duplica in diversi stati

Il linguaggio accettato dall'automa è l'insieme di tutte le stringhe sull'alfabeto tale che l'intersezione tra gli stati raggiunti e gli stati finali non sia vuota:

$$\mathcal{L}(M) = \{\sigma \in \Sigma^* | \hat{\delta}(q_0, \sigma) \cap F \neq \emptyset\}$$

Alla fine dell'intera lettura della stringa si ha n insieme di stati; se almeno uno di questi stati appartiene all'insieme degli stati finali, allora la stringa è accettata.

2.2.3 Automa a stati finiti non deterministico con ϵ-transizioni

Un automa a stati finiti non deterministico ϵ-transizioni (transizione senza leggere alcun simbolo) è definito come:

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle$$

In questo caso, la funzione di transizione δ è definita come:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q$$

Ciò siginifca che $\Rightarrow$ la macchina è in uno stato Q $\Rightarrow$ legge in input un simbolo di Σ ppure nulla $\{\epsilon\}$ $\Rightarrow$ questo va nelle parti di Q.

Ne consegue che anche $\hat{\delta}$ è. definita diversamente e occorre introdurre il concetto di **chiusura** ϵ di uno stato q.

Definizione: ϵ-chiusura

Se uno stato $p \in Q$, allora l'ϵ-**chiusura** di p è dato dall'insieme di tuti gli stati $q \in Q$ tali che da p, con un certo numero etichettati ϵ, permettono di arrivare a q.

$$\epsilon\text{-}chiusura(p) = \{q \in Q \mid p \xrightarrow{\epsilon^*} q\}$$

N.B. l'ϵ-chiusura è l'insieme di stati che vengo percorsi leggendo ϵ.

Per induzione si definisce $\hat{\delta}$ come:

$$\begin{cases} \hat{\delta}(q, \epsilon) = \epsilon\text{-}chiusura(q) & \text{se } \sigma = \epsilon \\ \hat{\delta}(q, xa) = \epsilon\text{-}chiusura\left(\bigcup_{p \in \hat{\delta}(q,x)} \delta(p, a)\right), & \text{se } \sigma = xa,\ a \in \Sigma \end{cases}$$

2.3 Espressioni regolari

Le **espressioni regolari** sono un modo compatto per rappresentare i linguaggi regolari.
Esse sono formate da simboli e operatori che permettono di costruire stringhe sull'alfabeto Σ.

Il significato delle espressioni regolari è dato dalla funzione **Exp** che associa a ogni espressione regolare e un linguaggio **Exp**(e).
Le espressioni regolari sono definite induttivamente come:

- se $\emptyset$ è l'insieme vuoto, allora $\emptyset$ è un'espressione regolare e **Exp**$(\emptyset) = \emptyset$
- se ϵ è la stringa vuota, allora ϵ è un'espressione regolare e **Exp**$(\epsilon) = \{\epsilon\}$
- se $a \in \Sigma$, allora a è un'espressione regolare e **Exp**$(a) = \{a\}$

Exp è un'epressione regolare definita induttivamente, la quale restituisce un linguaggio regolare, ovvero $2^{\Sigma}*$ (insieme di Σ*).

In questo modo, si possono definire i linguaggi sucessivi in modo induttivo:

- **linguaggio unione** $(+)$: rappresenta la scelta tra due stringhe

$$\textbf{Exp}(e_1 + e_2) = \text{Exp}(e_1) \cup \textbf{Exp}(e_2)$$

- **linguaggio concatenazione** $(\cdot)$: rappresenta la giustapposizione di due stringhe

$$\textbf{Exp}(e_1 \cdot e_2) = \textbf{Exp}(e_1) \cdot \textbf{Exp}(e_2)$$

- **linguaggio iterazione** $(^*)$: rappresenta la ripetizione di una stringa zero o più volte

$$\textbf{Exp}(e^*) = \textbf{Exp}(e)^*$$

2.3.1 Proprietà delle espressioni regolari

Nelle espressioni regolari valgono le seguenti proprietà:

Propierità delle espressioni regolari

- **distributiva sinistra**:

$$e_1 \cdot (e_2 + e_3) = (e_1 \cdot e_2) + (e_1 \cdot e_3)$$

- **distributiva destra**:

$$(e_1 + e_2) \cdot e_3 = (e_1 \cdot e_3) + (e_2 \cdot e_3)$$

- **associativa**:

$$e_1 + (e_2 + e_3) = (e_1 + e_2) + e_3$$

- **elemento neutro**:

$$\epsilon e = e\epsilon = e$$

- **elemento vuoto**:

$$\emptyset \cdot e = e \cdot \emptyset = \emptyset$$

- **unione di due linguaggi medesimi**:

$$e + e = e$$

- **iterazione del vuoto**:

$$\emptyset^* = \epsilon$$

- **assorbimento dell'iterazione di e***:

$$e^* + e = e^*$$

- **iterazione del'iterazione di e***:

$$(e^*)^* = e^*$$

- **iterazione della concatenazione**:

$$(e_1 \cdot e_2)^* = (e_1^* \cdot e_2^*)^*$$

- **unione di un linguaggio e del vuoto**:

$$e + \emptyset = e$$

2.3.2 Proprietà di chiusura dei linguaggi

In genere, i linguaggi regolari sono **chiusi**: ci sono delle operazioni che, applicate a due linguaggi regolari, generano un linguaggio regolare.
Le operazioni di chiusura sono le operazioni di **unione**, **iterazione** e **concatenazione**.

$$\cup, \cdot : \text{REG} \times \text{REG} \to \text{REG}$$

$$^* : \text{REG} \to \text{REG}$$

Altre proprietà derivano da questa osservazione: *se $\mathcal{L} \in REG \subseteq 2^{\Sigma}*$, allora per il complemento di $\mathcal{L}$, cioè $\bar{\mathcal{L}} = \Sigma^* \setminus \mathcal{L}$, esiste un ASFD tale che riconosca questo $\bar{\mathcal{L}}$?*

Quando si costruisce un automa deterministico M', occore rispettare la condizione per cui $\hat{\delta}(q_0, x) \notin F$.

Ogni stato non finale di un automa, che riconosce $\mathcal{L}$, è uno stato finale di un automa che riconosce il complemento di $\mathcal{L}$ ($\bar{\mathcal{L}}$):

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle \qquad \text{riconosce } \mathcal{L}$$

$$M' = \langle Q, \Sigma, \delta, q_0, F' \rangle \qquad \text{riconosce } \bar{\mathcal{L}}$$

dove $F' = Q \setminus F$.
Pertanto $\bar{\mathcal{L}}$ si può vedere come:

$$\bar{\mathcal{L}} = \Sigma^* \setminus \mathcal{L} = M'$$

Applicando le **legge di De Morgano** si ottiene che i linguaggi regolari sono **chiusi per intersezione**:

$$\mathcal{L}_1, \mathcal{L}_2 \in \text{REG} \Rightarrow \mathcal{L}_1 \cap \mathcal{L}_2 = \text{REG}$$

N.B.

Per De Morgan l'intersezione tra $\mathcal{L}_1$ e $\mathcal{L}_2$ equivale al complemento del comento di $\mathcal{L}_1$ unito al complemento di $\mathcal{L}_2$:

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \overline{\overline{\mathcal{L}_1} \cup \overline{\mathcal{L}_2}}$$

2.3.3 Corrispondenza tra espressioni regolari e ASFD

Teorema: McNaughton-Yamada (1960)

Sia r un'espresione regolare.
Allora esiste un ϵ-ASFND M tale che il linguaggio da esso riconosciuto sia uguale al linguaggio dall'espressione regolare r:

$$\exists M(\epsilon\text{-ASFND}) : \mathcal{L}(M) = \text{Exp}(r)$$

Il teorema afferma che ogni volta in cui si ha un'espressione regolare, allora si ha un linguaggio regolare:

$$\text{Exp.REG} \Leftrightarrow \text{Ling.REG}$$

2.4 Automa minimo

Per parlare di un **automa minimo** occorre introdurre alcuni concetti.

Definizione: partizione di un insieme S

Corrisponde a tanti sottoinisiemi $S_1, \ldots, S_n$ tali che:
- $\forall$ i = 1, . . . , n: $S_i \subseteq S$

- se si uniscono tutti gli S_i, allora si ottine S:

$$\bigcup_{i=1}^{n} S_i = S$$

- $\forall$ i $\neq$ j: $S_i \cap S_j = \emptyset$

Definizione: relazione di equivalenza

Se la relazione $\sim$ è di equivalenza, ossia è un sotoinsieme di $S \times S$, allora pre ogni $a \in S$ si ha che la classe di equivalenza di un oggeto, rispetto la relazione $\sim$, è un insieme $b \in S$ tale che a è equivalente a b:

$$[a]_\sim = \subseteq S$$

Se si considera la collezione di tutte le classi di equivalenza tale che $a \in S$, allora è una partizione di $S : \{[a]_\sim \mid a \in S\}$ è una partizione di S.

Definizione: indice finito

Si dice che la relazione $\sim$ sia l'indice finito quando ha una cardinalità finita, ovvero se partiziona l'indice S in un numero finito di clasi di equivalenza:

$$|\{[a]_\sim : a \in S\}| < \omega$$

Definizione: relazione raffinata di equivalenza

E' un raffinamento della relazione $\sim$, se per ogni classe di equivalenza $[a]_\approx$ questa è un sottoinisieme di $[b]_\sim$:

$$[a]_\approx \subseteq [b]_\sim$$

2.4.1 Relazioni tra linguaggi

Dato un linguaggio $\mathcal{L} \subseteq \Sigma^*$, si definisce la **relazione indotta dal linguaggio** $\mathcal{R}_\mathcal{L}$ come sottoinsieme di $\Sigma^* \times \Sigma^*$:

$$\mathcal{R}_\mathcal{L} \subseteq \Sigma^* \times \Sigma^*$$

Una parola x è in **relazione** $\mathcal{L}$ con una parola y se e solo se, per ogni parola $z \in \Sigma^*$, si aggiunge a destra di x la z e questa appartiene a $\mathcal{L}$ se e solo se la stessa cosa vale per y con l'aggiunta a destra di z:

$$x \, \mathcal{R}_\mathcal{L} \, y \;\Leftrightarrow\; \forall z \in \Sigma^* : xz \in \mathcal{L} \Leftrightarrow yz \in \mathcal{L}$$

Dato un atoma a stati finiti deterministico $M = \langle Q, \Sigma, \delta, q_0, F \rangle$, si definisce la **relazione indotta dall'automa** $\mathcal{R}_M$ come sottoinsieme di $\Sigma^* \times \Sigma^*$:

$$\mathcal{R}_M \subseteq \Sigma^* \times \Sigma^*$$

Una parola x è in **relazione** M con una parola y se e solo se la macchina, partendo dallo stato iniziale q_0 e leggendo x, arriva in uno stato q_x e leggendo y arriva in uno stato q_y tali che entrambi gli stati sono finali oppure entrambi non sono finali:

$$x \; \mathcal{R}_M \; y \;\; \Leftrightarrow \;\; \hat{\delta}(q_0, x) \Leftrightarrow \hat{\delta}(q_0, y)$$

Si può affermmare che entrambe le relazioni sono equivalenti.

Una relazione dei linguaggi $\mathcal{R} \subseteq \Sigma^* \times \Sigma^*$ è **invariante destra** se:

$$x \; \mathcal{R} \; y \Rightarrow \forall z \in \Sigma^* : xz \; \mathcal{R} \; yz$$

Lemma

Sia $\mathcal{R}_\mathcal{L}$ e sia $\mathcal{R}_\mathcal{M}$ sono invarianati destra.

2.4.2 Teorema di Myhill-Nerode

Teorema: Myhill-Nerdode (1960)

I seguenti tre enunciati sono equivalenti:

1. $\mathcal{L} \subseteq \Sigma^*$ è regolare
2. $\mathcal{L}$ è unione di classi di equivalenza su Σ^* indotte da una relazione invariante destra di indice finito
3. $\mathcal{R}_\mathcal{L}$ è di indice finito

2.5 Proprietà dei linguaggi regolari

Il **pumping lemma** afferma che se si ha un linguaggio regolare, allora esiste sempre un numero finito n tale che, per ogni stringa più lunga di questo n, è possibile spezzare la stringa z in tre sottostringhe uvw, in modo che:

- le prime due u e v hanno una lunghezza minore o uguale a n
- dentro v c'è almeno un simbolo
- si può ripetere la sottostringa v un numero arbitrario di volte e rimanere ancora in $\mathcal{L}$

Lemma: Pumping lemma (linguaggi regolari)

Sia $\mathcal{L} \subseteq \Sigma^*$ regolare.
Allora, esiste un numero $n \in \mathbb{N}$ tale che, per ogni stringa $z \in \mathcal{L}$, se $|z| \geq n$, allora esistono $u, , v, w \in \Sigma^*$ tali per cui:

1. la stringa z è suddivisa in $uvw : z = uvw$
2. la lunghezza di uv è minore o uguale a $n : |uv| \leq n$
3. la lunghezza di v è strettamente maggiore di $0 : |v| > 0 \Rightarrow$ in v è presente almeno un simbolo di Σ^*
4. per ogni i maggiore o uguale a 0, se si pompa v con i, allora continua a essere parte di $\mathcal{L}$:

$$\forall i \geq 0.uv^iw \in \mathcal{L}$$

Dimostrazione

Dato questo lemma, valgono le seguenti proprietà.

Proprietà pumping lemma

$\Rightarrow$ se $\mathcal{L}$ è regolare, allora vale il *pumping lemma* dei linguaggi regolari:

$$\mathcal{L} \in \text{REG} \Rightarrow \text{PL linguaggi regolari}$$

Quindi, esiste un numero $n \in \mathbb{N}$ tale che per ogni z appartenente alle stringhe Σ^*, se $z \in \mathcal{L}$ e $|z| \geq n$, allora esiste la stringa uvw appartenente alle stringhe Σ^* tale per cui $z = uvw$, $|uv \leq n$, $|u| > 0$ e $\forall i \in \mathbb{N} : uv^iw \in \mathcal{L}$:

$$\exists n \in \mathbb{N}.\forall z \in \Sigma^* : z \in \mathcal{L} \wedge |z| \geq n \Rightarrow \exists u, v, w \in \Sigma^*$$

$\Rightarrow$ se si nega il PL linguaggi regolari, allora $\mathcal{L}$ non è regolare:

$$\neg\text{PL linguaggi regolari} \Rightarrow \mathcal{L} \notin \text{REG}$$

Si nega l'espressione vista per la prima propierità:

$$\forall n \in \mathbb{N}.\exists z \in \Sigma^* : z \in \mathcal{L} \wedge |z| \geq u \wedge \forall n \wedge \forall u, v, w \in \Sigma^* : \neg(c_1 \wedge c_2 \wedge c_3 \wedge c_4)$$

$$: (c_1 \wedge c_2 \wedge c_3) \vee \neg c_4$$

$$: (c_1 \wedge c_2 \wedge c_3) \Rightarrow \neg c_4$$

$$: (z = uvw \wedge |uv| \neq n \wedge |v| > 0) \Rightarrow \exists : uv^iw \notin \mathcal{L}$$

N.B. Si ricorda che $A \Rightarrow B$ equivale a scrivere $\neg A \vee B$ e viceversa.

Per dimostrare, quindi, che un linguaggio non è regolare:

$\Rightarrow$ si suppone un numero n generico

$\Rightarrow$ si sceglie una stringa z dipendente da n e presente nel linguaggio

$\Rightarrow$ si dimostra che per ogni possibile suddivisione della stringa, in tre sottostringhe uvw, esiste un valore i tale per cui pompando/spompando v si esce dal linguaggio $\mathcal{L}$

Esempio:

Consideriamo il linguaggio $\mathcal{L} = \{a^n b^n \mid n \geq 0\}$ e per qualunque n si deve decidere se la stringa $a^n b^n \in \mathcal{L}$.
Tuttavia, se l'unica memoria a disposizione è lo stato, allora, per ricordare la precisa quantità di a e b incontrati occorre uno stato per ciascun simbolo, ma in questo modo diventerbbe un automa a stati finiti.

$\Rightarrow$ *$\mathcal{L}$ è regolare?*
Si prende un generico $n \in \mathbb{N}$ e si considera una stringa $a^n b^n$ dipendente da n e lunga maggiore o uguale a n:

$$a^n b^n \in \mathcal{L}, \quad |a^n b^n| \geq n$$

Per ogni possibile suddivisione in tre stringhe si deve avere:

$$|uv| \leq n$$

Infatti, uv non contiene elementi di b^n, ma solo : $uv = a^k$ con $k \leq n$.
$|v| > 0$, infatti, esiste un valore i per cui esiste un pompaggio che esce da $\mathcal{L}$:

$$i = 2 \quad \rightarrow \quad uv^2w = uvvw$$

Poichè $uv = a^k$, allora implica che $v = a^h$ con $h \leq n$.
Pertanto nella stringa pompata, il numero di a è strettamente maggiore del numero di b e $uv^2w \notin \mathcal{L}$.

Ecco dimostrata la negazione del pumping lemma:

$$\forall n \in \mathbb{N}.\exists z \in \Sigma^* : z \in \mathcal{L} \wedge |z| \geq n \wedge \forall u, v, w \in \Sigma^* : z = uvw \wedge |uv| \leq n \wedge |u| > 0 \Rightarrow \exists i : uv^2w \notin \mathcal{L}$$

Poichè è **dimostrata** la negazione del *pumping lemma*, allora $\mathcal{L} \in$ REG.

Esempio:

Consideriamo il linguaggio $\mathcal{L} = \{a^{n^2} \mid n \geq 1\}$ e per un qualunque n si deve decidere se $a^{n^2} \in \mathcal{L}$.
Tuutavia non si riesce ad avere abbastanza memoria, perchè è proporzionale alla crescita del numero.

$\Rightarrow$ *$\mathcal{L}$ è regolare?*
Consideriamo un generico $n \in \mathbb{N}$ e una stringa $z = a^{n^2}$ dipendente da n e lunga maggiore o uguale a n:

$$a^{n^2} \in \mathcal{L}, \quad |a^{n^2}| \geq n$$

Per ogni possibile suddivisione in tre sstringhe si deve avere $|uv| \neq n$ e un valore i tale che il pompaggio di v esca da $\mathcal{L}$.
Sapendo che:

$$|uv| = n^2 < |uv^2w| = |uvvw| = |uvwv| \neq n^2 + n < (n+1)^2$$

La stringa pompata uv^2w è compresa tra n^2 e $(n+1)^2$, di conseguenza la lunghezza di uv^2w non è un quadrato.
Di conseguenza $uv^2w \notin \mathcal{L}$.

Poichè è **dimostrata** la negazione del *pumping lemma*, allora $\mathcal{L} \notin$ REG.

3

Automi a pila